

Tema 04 · JavaScript y manejo del DOM

Clase 018 (19-08) · Teoría: «JavaScript» (Clase019-JS-WebServer.pdf)

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar
· 2.º cuatrimestre

1. Qué es JavaScript

JavaScript (JS) es un lenguaje de programación **interpretado** que se ejecuta **en el navegador del cliente** (no en el servidor). Se usa para validar datos, reaccionar a acciones del usuario (clics, escritura) y modificar la página sin recargarla. Es **débilmente tipado** (las variables no declaran tipo) y, pese al nombre, es distinto de Java aunque la sintaxis se parezca. Es independiente de la plataforma.

Cómo se integra con HTML: (a) como archivo externo `<script src="logica.js"></script>` (recomendado), o (b) código dentro de la etiqueta `<script>...</script>`. Se suele poner al final del body para que el HTML ya exista cuando el script lo busque.

2. Lo básico del lenguaje

Tema	Detalle
Variables	var (antigua, alcance de función), let (alcance de bloque, se puede reasignar), const (constante, no se reasigna). Hoy se usan let y const.
Tipos	number, string, boolean, null, undefined, object, function; arrays; conversión automática.
Operadores	Aritméticos + - * / %, comparación == (compara valor) y === (compara valor y tipo, preferido), lógicos && !
Condicionales y ciclos	if/else, switch, ternario (cond ? a : b), for, while, do...while, for...in, break/continue
Funciones	function nombre(a, b) { return a + b; } — sin declarar tipos; también funciones anónimas y flecha (x) => x * 2
Mensajes	alert("texto") (ventana), confirm() (aceptar/cancelar), prompt() (pide un dato), console.log() (consola del navegador, F12)
Arrays	let a = [1, 'x', true]; a.length, push, map, join, sort, reverse...

3. Eventos

JavaScript funciona por **eventos** (igual que Swing en Java): una fuente (botón, campo) genera un evento (click, submit, change...) y un escuchador (listener) lo atiende. Dos formas:

- En el HTML: `<button onclick="hacerClic()">` (simple, pero mezcla HTML con JS).
- En el JS: `elemento.addEventListener("click", function() { ... })` (separa responsabilidades, es la forma moderna).

Eventos frecuentes: **click**, **submit** (al enviar un formulario), **change**, **focus/blur**, **load** (al cargar la página), **keyup/keydown**.

4. El DOM (Document Object Model)

Cuando el navegador carga el HTML construye un **árbol de objetos** llamado **DOM**: cada etiqueta es un nodo. JavaScript puede leer y modificar ese árbol. Los objetos principales son **window** (la ventana, raíz de todo), **document** (la página), **location** (la URL) e **history**. Operaciones habituales:

Qué quiero hacer	Cómo
Buscar un elemento por su id	<code>document.getElementById("campoNombre")</code>
Leer lo que escribió el usuario	<code>input.value</code>
Cambiar el texto de un elemento	<code>elemento.textContent = "Hola"</code> (seguro) o <code>innerHTML</code> (interpreta HTML: cuidado con XSS)
Cambiar estilos	<code>elemento.style.color = "red"</code>
Vaciar un campo	<code>input.value = ""</code>
Evitar el comportamiento por defecto	<code>event.preventDefault()</code> (por ejemplo, impedir que el formulario se envíe)

5. Validación en el cliente

JavaScript permite validar un formulario **antes de enviarlo**: se escucha el evento **submit**, se revisan los campos y, si hay errores, se llama a `event.preventDefault()` y se muestran los mensajes. Da una respuesta inmediata al usuario y ahorra viajes al servidor.

Recordar: la validación en el navegador **no reemplaza** a la del servidor, porque el usuario puede desactivar JS o enviar peticiones manuales. Es solo mejora de experiencia (UX).

6. Bootstrap

Bootstrap es un framework de CSS/JS que da estilos listos (botones, formularios, tablas, grillas, tarjetas) y diseño **responsive** (se adapta a celular y escritorio). Se incluye con una etiqueta `<link>` a un CDN en el head. Clases usadas en esta clase: `container` (centra el contenido), `mb-3` (margen abajo), `form-label`, `form-control` (campos de texto), `form-select` (listas), `btn btn-primary` (botón azul), `text-center`.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 018 - 19-08: JavaScript, DOM y validación de un formulario de transferencia

Hay dos prácticas: (1) **index.html + logica.js**: variables, botón con onclick y manejo del DOM (saludar al usuario); (2) **transferencia.html + transferencia-logica.js**: formulario de transferencia bancaria con **Bootstrap** y **validación del lado del cliente** que muestra los errores en pantalla.

018 - Clase 19-08 / pom.xml

Qué es y para qué sirve: POM mínimo del proyecto (sin dependencias).

Cómo defenderlo / qué decir: La clase es solo HTML y JavaScript: los archivos van en src/main/resources.

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |     <modelVersion>4.0.0</modelVersion>
6 |
```

Líneas 1 a 6: Cabecera.

```
7 |     <groupId>edu.usal</groupId>
8 |     <artifactId>clase-19-08</artifactId>
9 |     <version>1.0-SNAPSHOT</version>
10 |
```

Líneas 7 a 10: Coordenadas: **clase-19-08**.

```
11 |     <properties>
12 |         <maven.compiler.source>11</maven.compiler.source>
13 |         <maven.compiler.target>11</maven.compiler.target>
14 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15 |     </properties>
16 |
```

Líneas 11 a 16: Java 11 y UTF-8.

```
17 | </project>
```

Línea 17: Cierre.

018 - Clase 19-08 / src / main / resources / index.html

Qué es y para qué sirve: Página de práctica del DOM: un botón con evento onclick y un campo para escribir el nombre y recibir un saludo.

Cómo defenderlo / qué decir: Muestra las **dos formas de asociar eventos**: atributo onclick en el HTML (botón «Hacer Clic») y listener en el archivo JS (botón «Saludar», identificado con un id). Los id son los «ganchos» con los que el JS encuentra los elementos. El script va al final del body para que los elementos ya existan.

```
1 | <!DOCTYPE html>
2 | <html lang="en">
3 | <head>
4 |     <meta charset="UTF-8">
5 |     <title>Clase 19-08</title>
6 | </head>
```

Líneas 1 a 6: DOCTYPE HTML5, raíz html, head con charset y title.

```

7 | <body>
8 | <!--Etiqueta HTML5 - semantica-->
9 | <header>
10 |   <h1>Clase 19-0 | JS</h1>
11 | </header>
12 |

```

Líneas 7 a 12: Inicio del cuerpo. Un comentario HTML y la cabecera **<header>** con el título.

```

13 | <main>
14 |   <button onclick="hacerClic()">Hacer Clic</button>
15 |   <br>
16 |   <h3>Manejo del DOM con JS</h3>

```

Líneas 13 a 16: Contenido principal. El primer botón tiene **onclick="hacerClic()"**: al hacer clic ejecuta la función JS `hacerClic`. **
** hace un salto de línea y luego un título «Manejo del DOM con JS».

```

17 |   <div>
18 |     <label for="campoNombre">Ingrese tu nombre:</label>
19 |     <input type="text" id="campoNombre" placeholder="Ej : Martin">
20 |
21 |     <button id="btnSaludar">Saludar</button>
22 |   </div>

```

Líneas 17 a 22: Bloque con el **label** (observar la errata `for="campoNombre"l`: hay una «l» de más), el **input de texto** con `id="campoNombre"` y el botón `id="btnSaludar"`. Esos **id** los usa el JS.

```

23 |   <hr>
24 |   <p id="msjSaludo"></p>
25 | </main>
26 |

```

Líneas 23 a 26: Línea separadora y un párrafo vacío **<p id="msjSaludo">**: es el lugar donde JS escribirá el saludo.

```

27 | <script src="logica.js">
28 | </script>
29 |
30 | </body>
31 | </html>

```

Líneas 27 a 31: Carga el archivo JavaScript externo **logica.js**, al final para que el DOM ya esté construido. Cierra `main`, `body` y `html`.

018 - Clase 19-08 / src / main / resources / logica.js

Qué es y para qué sirve: Lógica JavaScript de la página: declaración de variables, función de un botón y manejo del DOM con un listener.

Cómo defenderlo / qué decir: Los pasos del manejo del DOM que hay que saber: **(1)** obtener los elementos con `getElementById`; **(2)** registrar un listener con `addEventListener`; **(3)** leer `value`; **(4)** validar; **(5)** modificar el DOM con `textContent` y `style`. Se usa **textContent** y no `innerHTML` para escribir el nombre: así lo que escriba el usuario nunca se interpreta como HTML (evita XSS).

```

1 | alert('Hola Mundo con JS!');
2 | console.log('Esto es un log');
3 |

```

Líneas 1 a 3: **alert()** muestra una ventana emergente al cargar el archivo y **console.log()** escribe en la consola del navegador (F12).

```

4 | var identificarVar = 0;
5 | let identificadorLet = "";
6 | const IDENTIFICADORCONST = 'const';
7 |

```

Líneas 4 a 7: Tres formas de declarar variables: **var** (antigua), **let** (variable de bloque reasignable) y **const** (constante, por convención en MAYÚSCULAS). No se declara el tipo.

```

8 | /*
9 | let arreglo = ['Texto', 1, true, null, undefined, 'Ensalada de frutas'];
10 |
11 | for(let i = 0; arreglo.length; i++){
12 |     console.log(arreglo[i])
13 | }*/
14 |

```

Líneas 8 a 14: Código comentado: un array con tipos mezclados (texto, número, booleano, null, undefined) y un for para recorrerlo. Tiene un error: la condición del for es arreglo.length en lugar de i < arreglo.length.

```

15 | function hacerClic(){
16 |     alert('Hiciste clic sobre el boton');
17 | }
18 |

```

Líneas 15 a 18: Función invocada desde el atributo onclick del primer botón: muestra una alerta.

```

19 | const inputNombre = document.getElementById('campoNombre');
20 | const buttonSaludar = document.getElementById('btnSaludar');
21 | const parrafoSaludo = document.getElementById('msjSaludo');
22 |

```

Líneas 19 a 22: Selecciona elementos del DOM por su id y los guarda en constantes: el campo de nombre, el botón y el párrafo del saludo.

```

23 | buttonSaludar.addEventListener("click", function (){

```

Línea 23: Registra un **escuchador del evento click** en el botón Saludar: la función anónima se ejecuta cada vez que se hace clic.

```

24 |     let nombreIngresado = inputNombre.value;
25 |

```

Líneas 24 a 25: Lee el texto del campo con .value.

```

26 |     if(nombreIngresado.trim() === ''){
27 |         alert('Ingresa un nombre');

```

Líneas 26 a 27: Validación: trim() quita espacios; si queda vacío muestra una alerta pidiendo el nombre.

```

28 |     } else {
29 |
30 |         parrafoSaludo.textContent = 'Hola ¡' + nombreIngresado + '!, Bienvenido a la clase de JS';
31 |         parrafoSaludo.style.color = 'red';
32 |         inputNombre.value = '';
33 |     }
34 | });

```

Líneas 28 a 34: Si hay nombre: escribe el saludo en el párrafo con **textContent**, le pone color rojo con **style.color** y limpia el campo (value = "").

018 - Clase 19-08 / src / main / resources / transferencia.html

Qué es y para qué sirve: Formulario de «Nueva transferencia» con estilos de **Bootstrap**.

Cómo defenderlo / qué decir: Combina HTML + Bootstrap + JS: el diseño lo da Bootstrap (clases) y el comportamiento lo da transferencia-logica.js. Faltan action/method y name en los campos: es un ejemplo para practicar validación del lado del cliente, no un envío real al servidor. El **div#msjErrores** está vacío a propósito: el JS lo completa.

```

1 | <!DOCTYPE html>
2 | <html lang="en">
3 | <head>
4 |     <meta charset="UTF-8">

```

Líneas 1 a 4: DOCTYPE y head: **meta charset**.

```

5 |     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    |         integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">

```

Línea 5: Carga la hoja de estilos de **Bootstrap 5.3.8** desde un CDN (red de distribución). El atributo integrity verifica que el archivo no fue alterado y crossorigin habilita esa verificación.

```

6 | <title>Banco USAL - Nueva Transferencia</title>
7 | </head>

```

Líneas 6 a 7: Título de la pestaña.

```

8 | <body>
9 |

```

Líneas 8 a 9: Inicio del cuerpo.

```

10 | <div class="container">
11 |   <h1 class="mb-3 text-center">Formulario de transferencia bancaria</h1>
12 |

```

Líneas 10 a 12: container: contenedor centrado de Bootstrap. Título **h1** con mb-3 (margen inferior) y text-center.

```

13 |   <form id="formTransferencia">
14 |     <div class="mb-3">

```

Líneas 13 a 14: Formulario con **id** para que el JS lo encuentre.

```

15 |       <label class="form-label" for="cbuDestino">CBU Destino: </label>
16 |       <input type="text"
17 |         id="cbuDestino"
18 |         class="form-control">
19 |     </div>
20 |
21 |     <div class="mb-3">

```

Líneas 15 a 21: Campo **CBU destino:** texto con form-control (estilo Bootstrap). Cada bloque está en un div class="mb-3".

```

22 |       <label class="form-label">Monto a transferir: </label>
23 |       <input type="number"
24 |         class="form-control"
25 |         id="monto"
26 |         step="0.01"
27 |         placeholder="0.00">
28 |     </div>
29 |
30 |     <div class="mb-3">

```

Líneas 22 a 30: Campo **monto:** type="number", step="0.01" permite centavos, con placeholder.

```

31 |       <label class="form-label">Motivo :</label>
32 |       <select id="motivo" class="form-select">
33 |         <option value="-">Seleccione Motivo</option>
34 |         <option value="alquiler">Alquiler</option>
35 |         <option value="expensas">Expensas</option>
36 |         <option value="varios">Varios</option>
37 |       </select>
38 |     </div>
39 |

```

Líneas 31 a 39: Lista desplegable (form-select) con una opción inicial «Seleccione Motivo» de valor «-» (la que el JS toma como «no eligió»).

```

40 |       <button type="submit"
41 |         class="btn btn-primary">Realizar Transferencia</button>
42 |     </form>
43 |

```

Líneas 40 a 43: Botón **submit** azul (btn btn-primary) que dispara el evento **submit** del formulario.

```

44 |     <div id="msjErrores"></div>
45 |   </div>
46 |

```

Líneas 44 a 46: Div vacío donde el JS mostrará los errores.

```

47 |   <script src="transferencia-logica.js"></script>
48 | </body>
49 | </html>

```

Líneas 47 a 49: Carga el archivo de lógica al final del body.

018 - Clase 19-08 / src / main / resources / transferencia-logica.js

Qué es y para qué sirve: Validación del formulario de transferencia del lado del cliente.

Cómo defenderlo / qué decir: Patrón clásico de validación: **acumular errores en un arreglo**; si hay errores, **preventDefault()** cancela el envío y se los muestra; si no, se limpia el mensaje y el formulario se envía. **Para el parcial:** esta validación mejora la experiencia, pero el servidor debe repetirla. Nota de seguridad: aquí los mensajes son fijos y se arman con **innerHTML**; si incluyeran texto del usuario habría que usar **textContent** para evitar XSS.

```
1 | const divErrores = document.getElementById('msjErrores');
2 | const formulario = document.getElementById('formTransferencia');
3 |
```

Líneas 1 a 3: Obtiene el **div de errores** y el **formulario** del DOM por su id.

```
4 | formulario.addEventListener("submit", function (event){
5 |     let errores = [];
6 |
```

Líneas 4 a 6: Listener del evento submit: se ejecuta cuando el usuario intenta enviar el formulario. El parámetro **event** permite cancelarlo. Crea el arreglo vacío **errores**.

```
7 |     let inputCbu = document.getElementById('cbuDestino').value.trim();
8 |     let inputMonto = parseFloat(document.getElementById('monto').value);
9 |     let comboMotivo = document.getElementById('motivo').value;
10 |
```

Líneas 7 a 10: Lee los tres campos: el CBU (texto sin espacios con **trim()**), el monto (convertido a número con **parseFloat**) y el valor elegido en el combo.

```
11 |     if(inputCbu === "") {
12 |         errores.push("El CBU es obligatorio");
13 |     } else if(isNaN(inputCbu)) {
14 |         errores.push("El CBU debe de contener numeros");
15 |     }
16 |
```

Líneas 11 a 16: Validación del CBU: si está vacío agrega «obligatorio»; si no, con **isNaN** comprueba que sean números; los mensajes se agregan con **push**.

```
17 |     if(isNaN(inputMonto) || inputMonto <= 0){
18 |         errores.push("El monto debe de ser mayor a 0");
19 |     }
20 |
```

Líneas 17 a 20: Validación del monto: debe ser un número mayor que 0 (si el campo está vacío **parseFloat** devuelve NaN).

```
21 |     if(comboMotivo === "-"){
22 |         errores.push("Debe de seleccionar un motivo");
23 |     }
24 |
```

Líneas 21 a 24: Validación del motivo: no se puede dejar la opción inicial.

```
25 |     if(errores.length > 0) {
26 |         event.preventDefault();
27 |
28 |         divErrores.innerHTML = "<ul> " + errores.map(err => `<li>${err}</li>`).join('') + "</ul>"
```

Líneas 25 a 28: Si hay errores: **event.preventDefault()** cancela el envío y muestra una lista **** con un **** por error (con **map** se transforma cada texto en un **li** y con **join('')** se unen). Usa **template literals** (comillas invertidas) para insertar **\${err}**.

```
29 |     } else {
30 |         divErrores.innerHTML = "";
31 |     }
32 | });
```

Líneas 29 a 32: Si no hay errores, vacía el **div** y deja que el formulario se envíe.

Resumen para estudiar

- JS corre en el **cliente**; manipula el **DOM** y reacciona a **eventos**.
- Declarar con let/const; comparar con ===; funciones con function o flecha.
- Recetas: getElementById → addEventListener → .value → validar → textContent/style.
- Validar un formulario: escuchar **submit**, acumular errores, **preventDefault()** si hay errores.
- Bootstrap: link CDN + clases (container, form-control, btn btn-primary, mb-3).

Preguntas típicas

¿Qué diferencia hay entre var, let y const?

var tiene alcance de función y es antigua; let tiene alcance de bloque y se puede reasignar; const no se puede reasignar.

¿Para qué sirve event.preventDefault()?

Cancela la acción por defecto del evento; en un submit impide que el formulario se envíe al servidor, permitiendo mostrar los errores primero.

¿Es suficiente validar con JavaScript?

No. Es una mejora de experiencia; la validación obligatoria se hace en el servidor porque el cliente puede ser manipulado.